

Anexo - QA automatizado con agente de OpenClaw



”Porque el club es de los socios, y la gestión es de SocioUnido”

Plataforma inteligente que centraliza la gestión administrativa de los clubes, optimizando el uso de espacios compartidos, previniendo la morosidad y asegurando los accesos al estadio.

Alumno	Correo electrónico	Padrón
Ascencio, Felipe Santino	fascencio@fi.uba.ar	110675
Ghosn, Lautaro Gabriel	lghosn@fi.uba.ar	106998
Guerrero, Martín	mguerrero@fi.uba.ar	107774
Zielonka, Axel	azielonka@fi.uba.ar	110310

Jefe de Cátedra Mg. Ing. Carlos Fontela cfontela@fi.uba.ar

Tutor Mg. Ing. Gabriel Piñeiro gpineiro@fi.uba.ar

Cotutor Ing. Agustín Perrotta aperrotta@fi.uba.ar

Índice

1. QA automatizado con agente de OpenClaw	3
1.1. Introducción y objetivos	3
1.2. Uso de la herramienta OpenClaw	3
1.3. Definición de tareas y directrices del agente	3
1.4. Resultados obtenidos	4
1.4.1. Aplicación móvil PWA (para socios)	4
1.4.2. Microservicio de pagos	4
1.4.3. Microservicio de analíticas	5
1.4.4. Microservicio de autenticación	5
1.4.5. API Gateway	5
1.5. Acciones tomadas	6
1.5.1. Implementación del principio de mínimo privilegio y modelo Zero-Trust . .	6
1.5.2. Sanitización de entradas, codificación de rutas y protección de datos	6
1.5.3. Protección criptográfica contra ataques de análisis de tiempo	6
1.5.4. Fortalecimiento de los contratos de interfaz para interoperabilidad con IA .	6
1.6. Conclusión	6

1. QA automatizado con agente de OpenClaw

1.1. Introducción y objetivos

En el marco del desarrollo de SocioUnido, alcanzar la más alta calidad en los estándares de ciberseguridad y de interoperabilidad se ha planteado como un objetivo fundamental e innegociable. En la actualidad, el ecosistema tecnológico experimenta un crecimiento exponencial en el desarrollo de herramientas de Inteligencia Artificial que interactúan directamente con plataformas y productos web. Por este motivo, hacemos un énfasis especial en garantizar la interoperabilidad de nuestro sistema con futuras IAs. Para que SocioUnido se mantenga a la vanguardia de estos avances, resulta clave que la arquitectura, las interfaces y los endpoints estén preparados no solo para usuarios humanos, sino también para integrarse de manera fluida, lógica y segura con agentes autónomos de terceros.

1.2. Uso de la herramienta OpenClaw

Para llevar a cabo una validación rigurosa de estos estándares, implementamos **OpenClaw**, un agente de Inteligencia Artificial diseñado específicamente para el testing automatizado y el QA (Aseguramiento de la calidad) integral de repositorios de código.

La adopción de OpenClaw se justifica por su capacidad innovadora para realizar auditorías exhaustivas sobre grandes volúmenes de código estructural. A diferencia de las herramientas de análisis estático tradicionales, OpenClaw comprende el contexto del código, identifica vulnerabilidades lógicas complejas y evalúa la preparación del sistema para la automatización, actuando como un auditor experto dedicado exclusivamente a las necesidades de SocioUnido.

1.3. Definición de tareas y directrices del agente

La implementación técnica del agente se basó en un marco de instrucciones precisas (prompting) que define su rol como *Experto en QA y Ciberseguridad*. Las directrices configuradas para su análisis auditan estrictamente tres ejes fundamentales:

- **Seguridad de la implementación:** Se instruye al agente a realizar una búsqueda exhaustiva de vulnerabilidades, haciendo foco en los estándares de la industria (como el OWASP Top 10) y en la prevención de vectores de ataque tanto típicos como atípicos.
- **Prevención de fuga de datos:** El agente debe asegurar rigurosamente que el código fuente no exponga credenciales, contraseñas, tokens de acceso ni información privada de los usuarios o de la institución. Este punto es considerado crítico, dada la naturaleza del sistema y su enfoque en ser comercializado a terceros.
- **Preparación para IA y automatización:** Se evalúa meticulosamente si la estructura de los datos y los endpoints del sistema están diseñados garantizando una interacción sin fricciones con futuras automatizaciones e Inteligencias Artificiales externas.

Para optimizar la legibilidad y eficiencia de la auditoría, se le exige al agente un formato de salida estricto. El reporte debe omitir el código limpio y detallar únicamente los archivos que presentan problemas o áreas de mejora, indicando para cada caso: el nombre del archivo, el problema detectado, la justificación de su impacto y la propuesta concreta de solución. En caso de no detectar fallas, el agente está instruido para emitir una única declaración certificando que la implementación cumple con todos los estándares definidos.

1.4. Resultados obtenidos

A continuación, se detalla el análisis exhaustivo de seguridad realizado por OpenClaw sobre los cinco repositorios principales que conforman la arquitectura de SocioUnido.

1.4.1. Aplicación móvil PWA (para socios)

- **Fuga de información sensible:** Se identificó que ante fallos inesperados de la aplicación, el componente encargado de capturar errores exponía la traza completa del sistema (stack trace) directamente en la pantalla del usuario, revelando la estructura interna del código fuente.
- **Manipulación de rutas API:** Se detectó una concatenación directa e insegura de parámetros en las peticiones HTTP enviadas por los servicios del cliente, dejando abierta la posibilidad de alterar las rutas de la interfaz de programación (Path Traversal) o enviar solicitudes malformadas.
- **Gestión de encabezados de autorización:** El módulo central de peticiones HTTP enviaba encabezados de autenticación con valores nulos o indefinidos cuando el usuario navegaba de forma anónima, generando un comportamiento ambiguo en los servidores.
- **Exposición de datos personales y tokens de notificación:** El contexto global de autenticación registraba en la consola del navegador información personal identificable del socio junto con los tokens del servicio de notificaciones, vulnerando la privacidad del usuario.
- **Configuración de entorno e integración de pagos:** Se detectó el hardcodeo de claves de prueba para la pasarela de pagos.
- **Metadatos del documento:** El archivo raíz de la interfaz mantenía el atributo de idioma configurado en inglés para una aplicación desarrollada íntegramente en español, lo que afectaba la accesibilidad y la interpretación de agentes externos.

1.4.2. Microservicio de pagos

- **Omisión de autenticación en procesamiento de cobros:** El punto de acceso principal para ejecutar transacciones financieras no aplicaba la validación de la clave interna del sistema, permitiendo a cualquier actor invocar operaciones de cobro de forma anónima.
- **Ausencia de verificación en notificaciones externas:** El servicio procesaba las notificaciones de pago entrantes desde la pasarela externa sin validar su firma criptográfica, exponiendo la plataforma a ataques de saturación o denegación de servicio (DoS).
- **Inseguridad en la comunicación con la base de datos en memoria:** La conexión hacia el intermediario de eventos (Redis) desactivaba la validación de certificados SSL/TLS, abriendo las transacciones financieras a intercepciones de red o ataques de tipo Man-in-the-Middle.
- **Gestión de credenciales en scripts de migración:** Se hallaron claves secretas débiles asignadas por defecto y cadenas de conexión con contraseñas expuestas dentro de los archivos de configuración de la base de datos.

1.4.3. Microservicio de analíticas

- **Procesamiento deficiente de mensajes asíncronos:** El receptor de eventos financieros leía los datos entrantes del bus de mensajería sin realizar una validación de estructura previa, lo que podía ocasionar fallos críticos e inconsistencias en las métricas ante un mensaje malformado.
- **Autenticación vulnerable a análisis de tiempo:** El mecanismo de verificación de tokens utilizaba comparaciones de texto convencionales, lo que permitía a un atacante inferir la clave secreta mediante el análisis estadístico de los tiempos de respuesta (Timing Attack).
- **Contratos de interfaz incompletos:** La falta de especificación formal para las respuestas de error en la documentación técnica (OpenAPI) dificultaba la integración automatizada por parte de clientes externos o agentes de Inteligencia Artificial.

1.4.4. Microservicio de autenticación

- **Políticas de contraseñas laxas:** El proceso de registro de nuevos usuarios permitía claves de acceso demasiado cortas, incumpliendo con las recomendaciones internacionales de seguridad para la gestión de identidades.
- **Uso de estructuras de datos genéricas:** Los controladores procesaban la información entrante mediante diccionarios genéricos sin tipo, lo que impedía la autogeneración de esquemas claros y bloqueaba la interacción de herramientas automatizadas que requieren conocer la estructura exacta de los datos.
- **Fuga de detalles técnicos en errores de registro:** Ante un fallo no controlado durante el alta de un usuario, el sistema devolvía al cliente el mensaje de excepción crudo del motor de base de datos o del proveedor de identidad, exponiendo detalles internos de la infraestructura.

1.4.5. API Gateway

- **Elevación de privilegios y falta de control de accesos:** Módulos sensibles, como la administración de roles o el enrolamiento de accesos físicos, carecían de verificación de permisos, permitiendo que usuarios con credenciales básicas ejecutaran acciones administrativas. Asimismo, el enrutador adjuntaba su clave secreta interna sin validar previamente el nivel de acceso del cliente.
- **Ausencia de límites de tráfico y políticas permisivas:** Diversas rutas expuestas carecían de mecanismos para restringir la tasa de peticiones por minuto (Rate Limiting) y utilizaban configuraciones de origen cruzado (CORS) demasiado laxas para entornos de producción.
- **Ejecución de contenedores con privilegios máximos:** El entorno de empaquetado del enrutador ejecutaba el servicio bajo el usuario principal del sistema operativo (**root**), lo que incrementaba el impacto en caso de un compromiso del contenedor.
- **Desactivación de la validación de llaves públicas:** La configuración desactivaba explícitamente la verificación de seguridad al descargar las llaves criptográficas del proveedor de identidades, permitiendo la eventual suplantación de firmas de autenticación.

1.5. Acciones tomadas

Las observaciones emitidas por el agente automatizado fueron integradas al flujo de trabajo mediante un proceso de refactorización integral en todos los módulos de la aplicación. A continuación, se detallan las correcciones aplicadas y sus fundamentos técnicos.

1.5.1. Implementación del principio de mínimo privilegio y modelo Zero-Trust

Se reestructuró el enrutamiento de peticiones para eliminar la adjunción automática de claves internas. Toda solicitud, tanto pública como entre servicios, se somete a una validación estricta de tokens de identidad, verificando criptográficamente el rol del emisor y otorgando únicamente los permisos indispensables para la acción requerida. Además, se reconfiguraron los entornos empaquetados en contenedores para ejecutarse bajo usuarios de sistema con privilegios acotados.

1.5.2. Sanitización de entradas, codificación de rutas y protección de datos

Se estandarizó la preparación de peticiones HTTP en el cliente web, aplicando codificación de caracteres a todo parámetro dinámico (como documentos o identificadores) previo a la conformación de la URL, anulando intentos de manipulación de rutas. La exposición no deseada de información se neutralizó suprimiendo la impresión de variables en consola durante la producción y reemplazando las trazas de error en pantalla por mensajes genéricos orientados al usuario final. Adicionalmente, las conexiones internas de datos fueron forzadas a exigir certificados SSL/TLS válidos.

1.5.3. Protección criptográfica contra ataques de análisis de tiempo

En los módulos de validación de seguridad de los microservicios se reemplazaron las comparaciones de texto tradicionales por funciones de comparación de tiempo constante. Este método garantiza que la verificación de una clave requiera exactamente la misma fracción de segundo independientemente de si los caracteres coinciden o no, eliminando la posibilidad de inferir el secreto mediante mediciones estadísticas de respuesta.

1.5.4. Fortalecimiento de los contratos de interfaz para interoperabilidad con IA

Para facilitar la interacción fluida con herramientas externas y agentes de Inteligencia Artificial, se eliminó el uso de estructuras de datos genéricas en los controladores. Todos los datos entrantes y salientes fueron modelados mediante esquemas estrictamente tipados y validados, incluyendo la definición explícita de las respuestas de error y la validación previa de los mensajes asíncronos que ingresan al sistema.

1.6. Conclusión

La integración de la herramienta OpenClaw como agente de aseguramiento de calidad demostró ser un activo estratégico para el proyecto. El análisis automatizado no solo permitió detectar fallas de configuración habituales en el desarrollo web, sino que también identificó vulnerabilidades lógicas complejas —como el análisis de tiempo en la validación de seguridad o configuraciones permisivas en canales de eventos— que suelen ser pasadas por alto en las revisiones de código tradicionales. Gracias a estas correcciones, SocioUnido cuenta con una arquitectura robusta, segura y técnicamente preparada para integrarse de forma transparente con futuras plataformas y agentes inteligentes.